

Absolutely. Let's start Phase 1.

We are going to deploy **Conduit**, which is the ultra-lightweight Matrix server written in Rust. It is famous for using less than 50MB of RAM, making it perfect for your Pi.

Here is the complete, A-to-Z cheatsheet for setting up your private Matrix server.

Prerequisites: Cloudflare Tunnel Preparation

Before touching the Pi, you need a domain name for your chat server so your family can connect to it from outside the house.

Since you already use Cloudflare, we will route this through a tunnel.

1. Go to your Cloudflare Zero Trust Dashboard.
2. Go to **Networks > Tunnels** and click your existing Homelab tunnel.
3. Click **Configure > Public Hostname > Add a public hostname**.
4. Set the **Subdomain** to chat (or matrix). Your full domain will be something like chat.redwanshub.studio.
5. Set the **Service Type** to HTTP.
6. Set the **URL** to 192.168.0.40:6167. *(We will configure Conduit to listen on port 6167).*
7. Click **Save**.

Step 1: Create the Folder Structure on the Pi

We need to create a dedicated folder for the Matrix server and its database. We will map the database to your external CCTV SSD to protect your OS NVMe drive, just like we did with Nextcloud and Firefly.

Open your Raspberry Pi terminal and run:

```
Bash
mkdir -p ~/homelab/matrix
cd ~/homelab/matrix
```

Step 2: Create the Conduit Configuration File

Conduit requires a configuration file to know its domain name and how to handle database writes.

1. Create a conduit.toml file:
Bash
nano conduit.toml
2. Paste this configuration inside. **Make sure to change chat.redwanshub.studio to your actual domain name.**
Ini, TOML
[global]
The name of your server (this will be the second half of your Matrix ID:
@nabil:chat.redwanshub.studio)

```
server_name = "chat.redwanshub.studio"
```

```
# The database backend (RocksDB is extremely fast and low-memory)
```

```
database_backend = "rocksdb"
```

```
# Where to save the database inside the container
```

```
database_path = "/var/lib/matrix-conduit/"
```

```
# Server port
```

```
port = 6167
```

```
# Allow users to register (Set to false after your family creates their accounts)
```

```
allow_registration = true
```

```
# Registration Token (Optional, but good for security. Remove the '#' below and set a secret word)
```

```
# registration_token = "MySecretFamilyCode"
```

3. Save and exit (Ctrl + O, Enter, Ctrl + X).

Step 3: Create the Docker Compose File

Now we will create the Docker stack.

1. Create the compose file:

```
Bash
```

```
nano docker-compose.yml
```

2. Paste this code:

```
YAML
```

```
services:
```

```
  conduit:
```

```
    image: matrixconduit/matrix-conduit:latest
```

```
    container_name: matrix_server
```

```
    restart: always
```

```
    ports:
```

```
      - "6167:6167"
```

```
    volumes:
```

```
      # Mount the config file we just made
```

```
      - ./conduit.toml:/etc/matrix-conduit/conduit.toml
```

```
      # Mount the database directory directly to the external SSD to protect NVMe
```

```
      - /mnt/cctv_ssd/matrix_db:/var/lib/matrix-conduit
```

```
    environment:
```

```
      - CONDUIT_CONFIG=/etc/matrix-conduit/conduit.toml
```

```
    deploy:
```

```
      resources:
```

```
        limits:
```

memory: 150M # Very strict memory limit (It usually only needs 50M)

3. Save and exit (Ctrl + O, Enter, Ctrl + X).

Step 4: Fix SSD Permissions

Because we are mounting the database to /mnt/cctv_ssd/matrix_db, we need to make sure the Conduit Docker container has the Linux permissions to write to that folder. Conduit runs internally as a specific user.

Run this command to create the folder on the SSD and grant full permissions:

```
Bash
sudo mkdir -p /mnt/cctv_ssd/matrix_db
sudo chown -R 1000:1000 /mnt/cctv_ssd/matrix_db
sudo chmod -R 777 /mnt/cctv_ssd/matrix_db
```

Step 5: Start the Matrix Server

Launch the container:

```
Bash
docker compose up -d
```

You can check the logs to make sure it started correctly without errors:

```
Bash
docker logs matrix_server
```

You should see a message saying "Starting Conduit on 0.0.0.0:6167"

Step 6: Connect via the Element App

Now that your server is running and Cloudflare is routing traffic to it, you can create your first account.

1. Download the **Element** app from the Google Play Store or Apple App Store on your phone.
2. Open the app and click **Get Started**.

3. **CRITICAL STEP:** Do not create an account on the default "matrix.org" server. Click **Edit** or **Custom Server** next to the server address.
4. Enter your custom domain:
https://chat.redwanshub.studio
5. Click **Create Account**.
6. Choose a username (e.g., nabil) and a strong password.
7. *Note: If you set a registration_token in the conduit.toml file, it will ask for that secret code now.*
8. Once logged in, the app will ask you to **"Set up Secure Backup"**. This is the End-to-End Encryption (E2EE) key setup. **Do this immediately.** Save the recovery key securely (like in your password manager).

What's Next?

Have a family member download the app, change the server to your URL, and create an account. You can now search for their username (e.g., @brother:chat.redwanshub.studio) and send them a fully encrypted text message!

Once you confirm texting is working flawlessly, let me know, and we will move to **Phase 2 (Installing Coturn)** so you can unlock high-quality voice and video calling.

This is Phase 2: The Video & Audio Routing Engine.

Before we begin, there is one critical networking rule you must know: **Video calls use UDP traffic, and Cloudflare Tunnels only support TCP/HTTP.**

Because of this, we cannot route your video calls through your Cloudflare tunnel. We have to open a direct, secure port on your Tenda router specifically for the Coturn relay server.

Here is the complete A-to-Z setup to build your private TURN server for flawless, encrypted WebRTC video calls.

Step 1: The Networking Prerequisite (Port Forwarding)

You must open ports on your Tenda router to allow the encrypted video packets to reach your Raspberry Pi.

8. Log into your Tenda Router (192.168.0.1).
9. Go to **Advanced** -> **Port Forwarding** (or Virtual Server).
10. Add the following rules pointing to your Raspberry Pi's IP (192.168.0.40):
 - **Rule 1:** Port 3478 (Set Protocol to Both or TCP & UDP).
 - **Rule 2:** Ports 50000-50050 (Set Protocol to UDP only). *This is the restricted port range we will use for the video streams.*

Step 2: Create the Coturn Directory

Open your Raspberry Pi terminal and create a dedicated folder for Coturn:

```
Bash
mkdir -p ~/homelab/coturn
cd ~/homelab/coturn
```

Step 3: Generate a Master Secret Key

Coturn requires a highly secure authentication password so random internet bots cannot use your server to relay their own traffic.

Run this command to generate a random secure key. **Copy the output; you will need it in the next steps.**

```
Bash
openssl rand -hex 32
```

Step 4: Create the Coturn Configuration File

Create the config file:

```
Bash
nano turnserver.conf
```

Paste the following code. (Replace YOUR_SECRET_KEY_HERE with the key you just generated, and YOUR_PUBLIC_IP_HERE with your actual public IP address from your ISP. You can find it by googling "What is my IP"):

```
Plaintext
# Basic Settings
listening-port=3478
```

```
fingerprint
use-auth-secret
static-auth-secret=YOUR_SECRET_KEY_HERE
realm=chat.redwanshub.studio
total-quota=100
bps-capacity=0
stale-nonce
no-multicast-peers

# Public IP binding (Required for the firewall bypass)
external-ip=YOUR_PUBLIC_IP_HERE

# Restricted UDP Port Range (To save Pi CPU/RAM)
min-port=50000
max-port=50050

# SSD Protection: Disable heavy logging to protect NVMe
log-file=/dev/null
```

Save and exit (Ctrl + O, Enter, Ctrl + X).

Step 5: Create the Docker Compose File

Create the stack for Coturn:

```
Bash
nano docker-compose.yml
```

Paste this configuration:

```
YAML
services:
  coturn:
    image: coturn/coturn:latest
    container_name: coturn_server
    restart: always
    network_mode: "host" # We use host mode so Docker doesn't bottleneck the UDP video packets
    volumes:
```

```
- ./turnserver.conf:/etc/coturn/turnserver.conf
deploy:
resources:
limits:
memory: 100M # Coturn is very lightweight
```

Save and exit, then start the server:

```
Bash
docker compose up -d
```

Step 6: Connect Matrix (Conduit) to Coturn

Now we must tell your Matrix chat server that the video relay station exists, so it can instruct the Element apps on your phones to use it.

4. Edit your Conduit configuration:

```
Bash
nano ~/homelab/matrix/conduit.toml
```

5. Scroll to the bottom of the [global] section and add these two lines (replace the IP and Secret to match Step 4):

```
Ini, TOML
turn_uris = ["turn:YOUR_PUBLIC_IP_HERE:3478?transport=udp",
"turn:YOUR_PUBLIC_IP_HERE:3478?transport=tcp"]
turn_secret = "YOUR_SECRET_KEY_HERE"
```

6. Save and exit (Ctrl + O, Enter, Ctrl + X).
7. Restart the Matrix server to apply the changes:

```
Bash
cd ~/homelab/matrix
docker compose restart
```

Step 7: Test Your Ultimate Privacy Server

Your Phase 1 and Phase 2 are now perfectly linked.

9. Disconnect your phone from your home Wi-Fi and switch to 4G/Mobile Data (to force it outside the local network).
10. Open the Element app.
11. Start a video call with a family member.

The Matrix server will connect the call, and Coturn will instantly bounce the encrypted UDP video packets off your Pi and through the mobile networks. You now have a fully operational,

enterprise-grade, End-to-End Encrypted communication platform running entirely on your own hardware!